

CP020003

ARTIFICIAL INTELLIGENCE



Khon Kaen University · 2026

Week 04

DNA Classification

Supervised Learning meets Deep Learning
— PyTorch Edition

1



Instructor: Teerapong Panboonyuen (P'Kao)

https://github.com/kaopanboonyuen/CP020003_ArtificialIntelligence_2026s1

What We'll Cover



Bioinformatics 101

DNA, genome, chromosome, protein — the vocabulary you need



This Week's Dataset

3,000 synthetic DNA sequences, 4 organism classes



Recap: scikit-learn

Decision Tree, Random Forest, XGBoost — and how to actually tune them



Recap: Deep Learning

Layers, activations, and the CNN motif-detector from last time



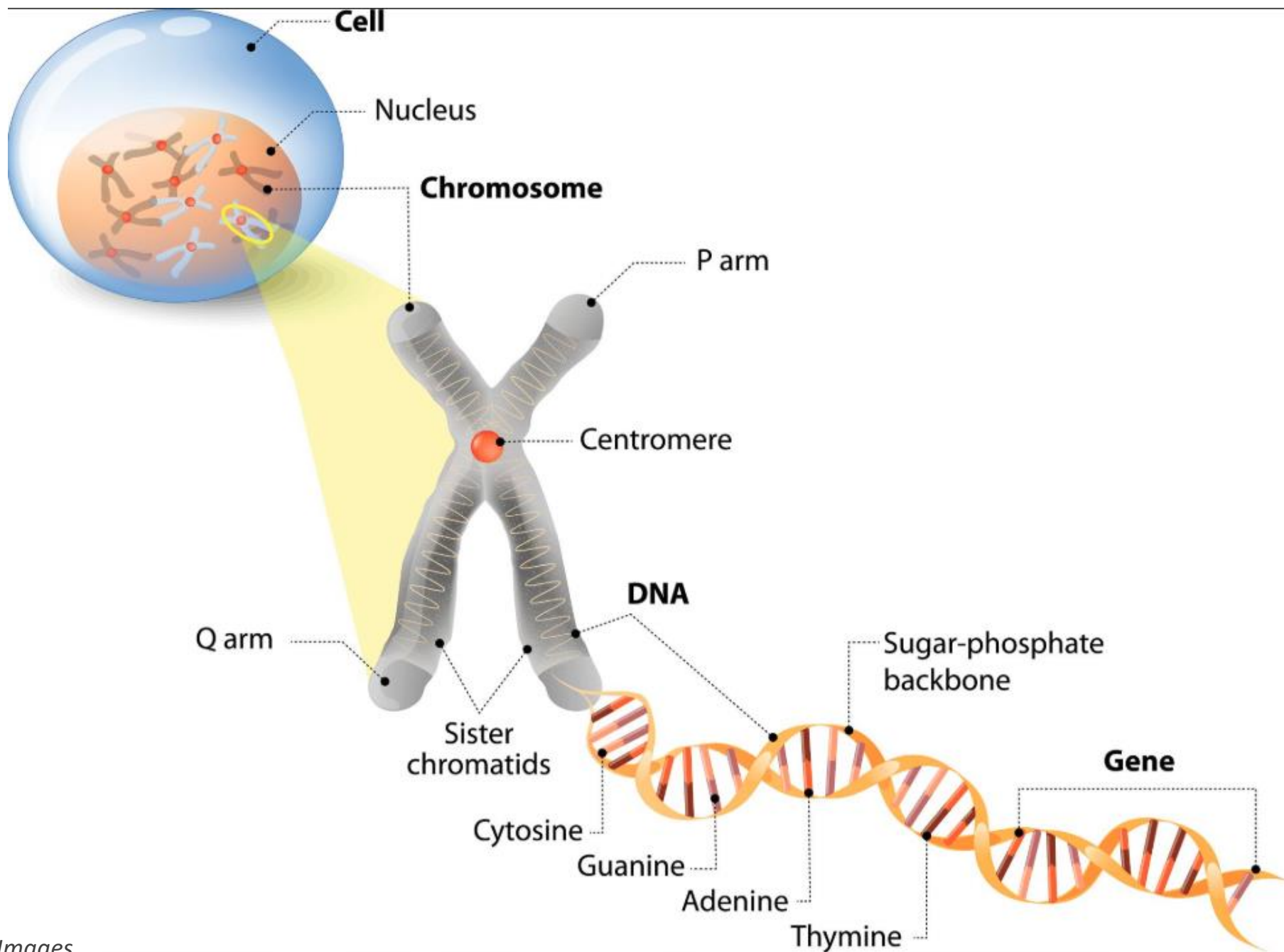
New: LSTM & GRU

Reading DNA sequentially — what they are and when to use them

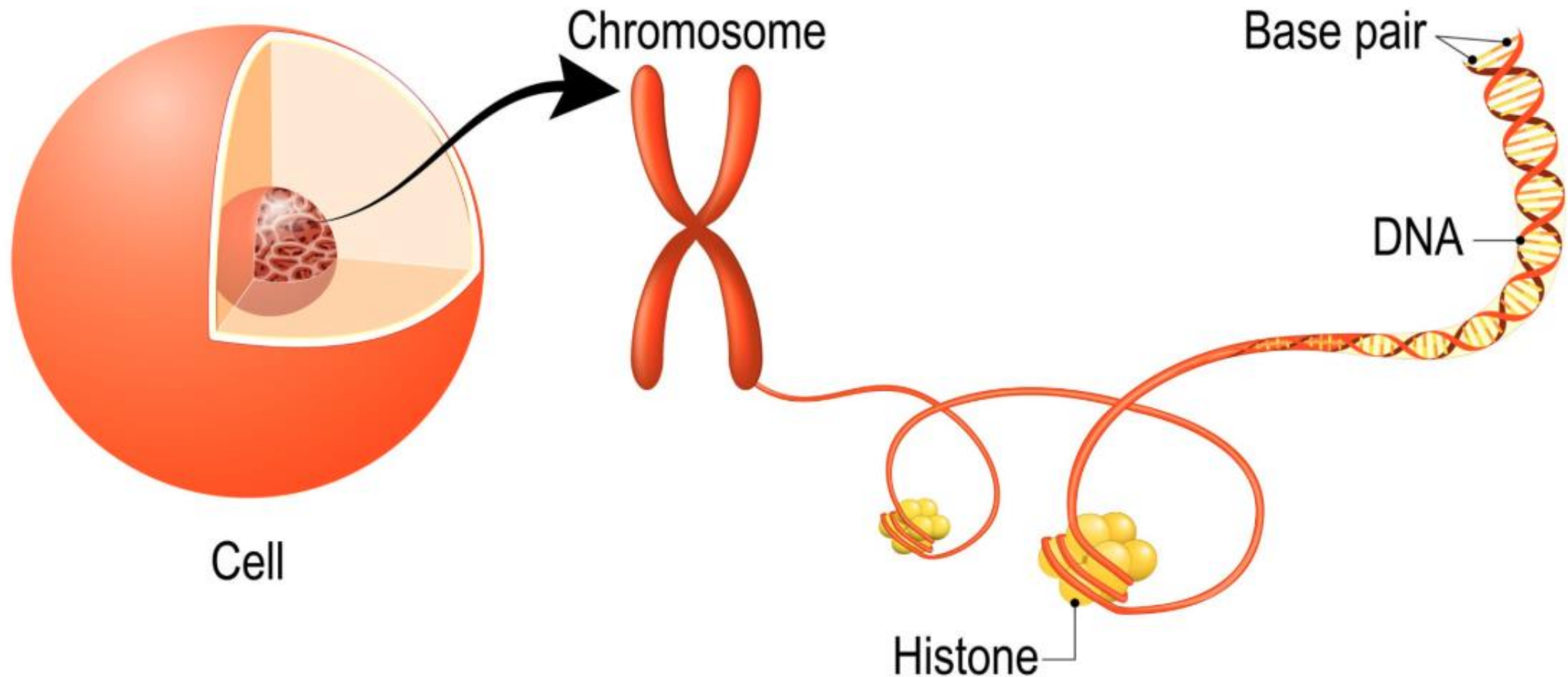


Results & Homework

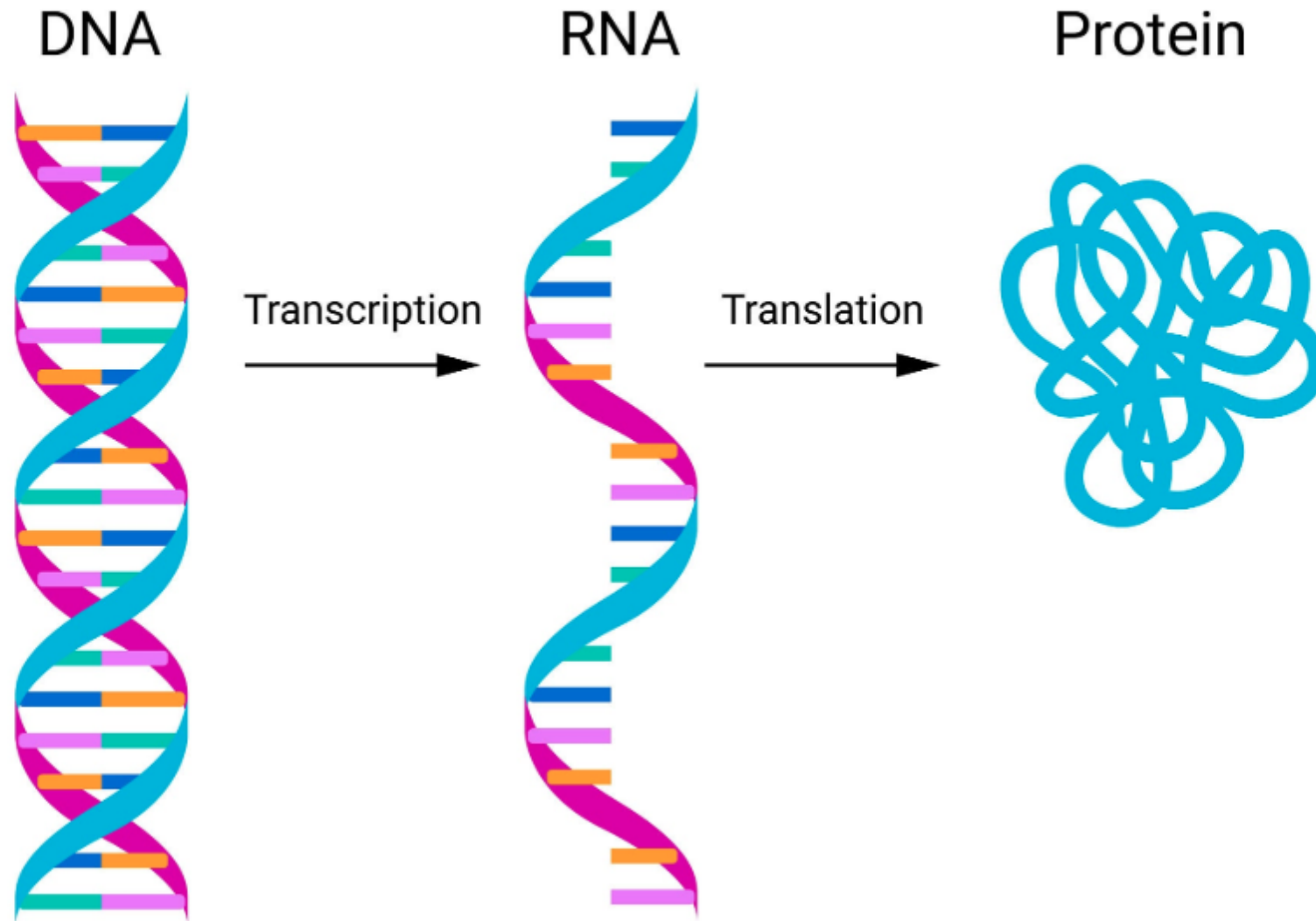
Full model leaderboard, plus a real bioinformatics dataset to take home



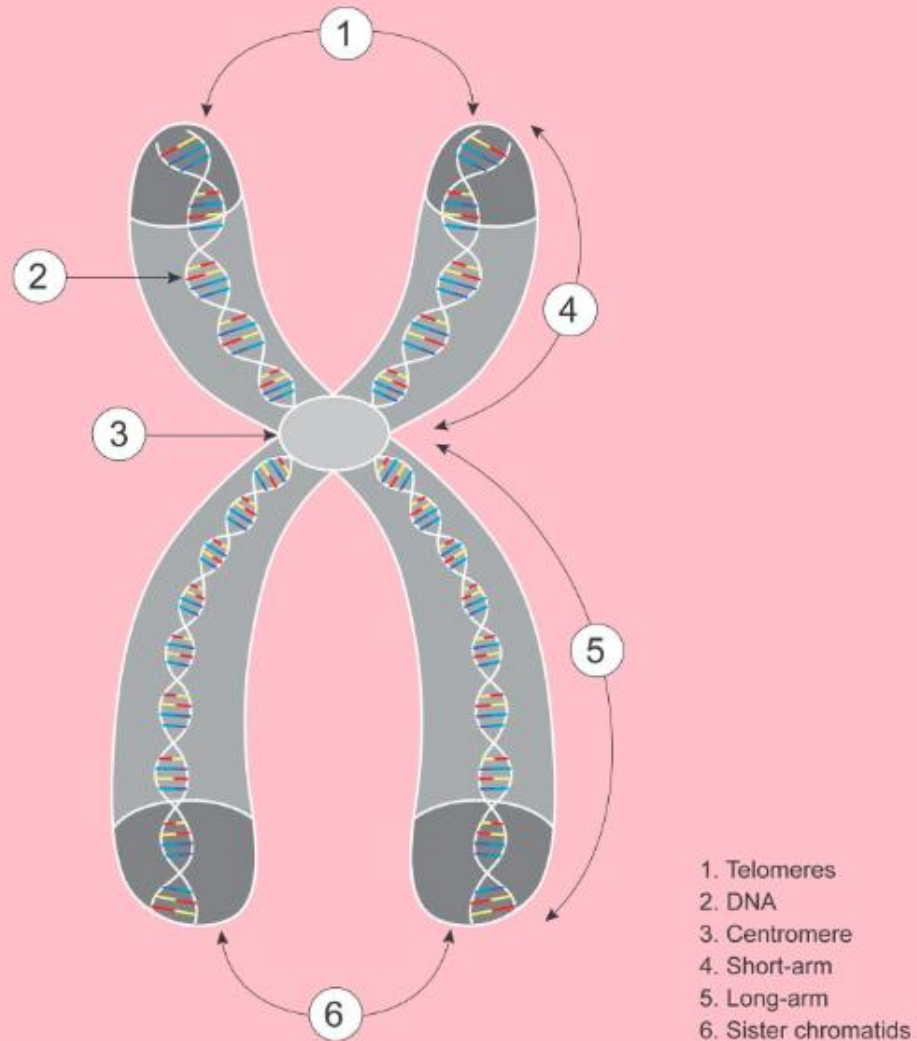
Chromosome structure and organization



DNA, RNA and protein synthesis

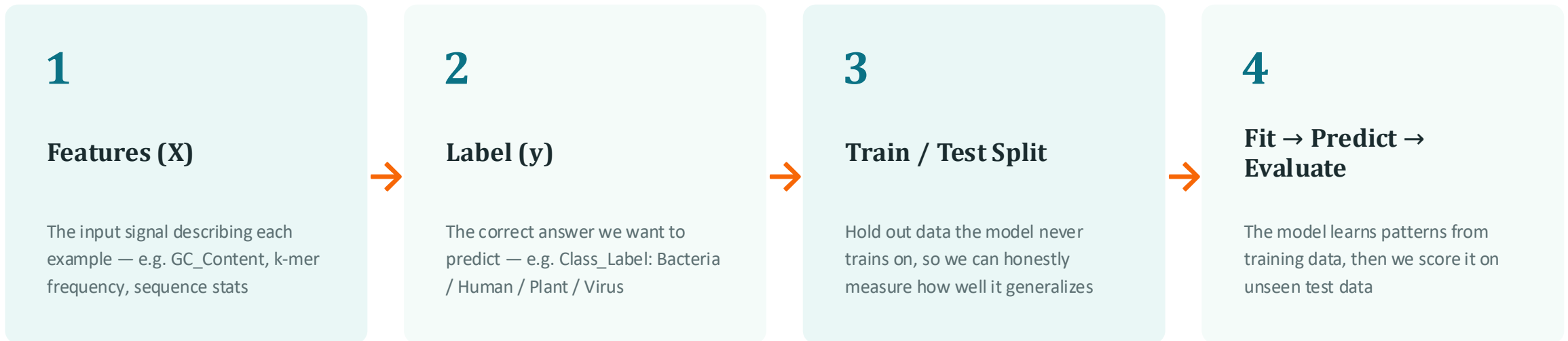


Chromosome Structure



What Is Supervised Learning, Again?

We give the model labeled examples — inputs paired with the correct answer — and it learns a function that maps one to the other.



This week: same core idea, brand-new input type — raw DNA sequences instead of a plain feature table.

DNA: Just a String Over 4 Letters

You don't need a biology degree for this — a few definitions, explained the way a programmer would explain them.

A

Adenine

C

Cytosine

G

Guanine

T

Thymine

Nucleotide = one of these 4 building blocks

CS analogy: one character in a 4-letter alphabet

{A,C,G,T}

DNA Sequence = a long string made of those 4 letters

"ACGGTTACCTGATCGGATCCA . . . "

Mutation = a letter flips, gets deleted, or inserted — like a code change or commit. Could be harmless, could break something critical.

Genome & Chromosome



Genome

Biology: The ENTIRE DNA sequence of an organism (billions of letters for humans!)

CS analogy: The full source code / codebase of an organism



Chromosome

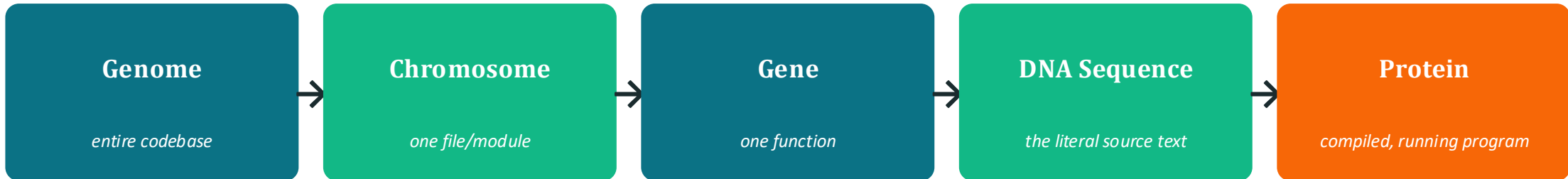
Biology: A tightly-packed bundle of DNA. Humans have 46 (23 pairs) holding the whole genome

CS analogy: Splitting one giant codebase into separate files or modules



🧠 *Quick check: if a genome is the whole codebase and a chromosome is one file, what's a single line of that file? → A nucleotide (one A, C, G, or T)!*

Gene, Protein & the Central Dogma



Gene = a meaningful sub-sequence that codes for something specific, like a protein (a function inside the codebase)

Protein = built by reading a gene, 3 letters (a codon) at a time, into a chain of amino acids (the compiled output your code produces when it runs)

Central Dogma

DNA → RNA → Protein — DNA is "transcribed" into RNA, then RNA is "translated" into protein: source code → intermediate representation → running executable.

Healthcare, Mutations & Real Motifs



Precision Healthcare Era

Every virus, bacterium, plant, and human cell carries its own DNA. When DNA mutates, it can create new disease variants — think of how often you hear about a new COVID "variant."



What Are Motifs?

Short, recurring DNA patterns with real biological meaning — like a recognizable syntax pattern in code (e.g. `def ...()`: always marking a function).

GAATTC

EcoRI restriction-enzyme site

→ *Bacteria*

TATAAA

TATA box (promoter element)

→ *Human*

CCAAT

CAAT box (promoter element)

→ *Plant*

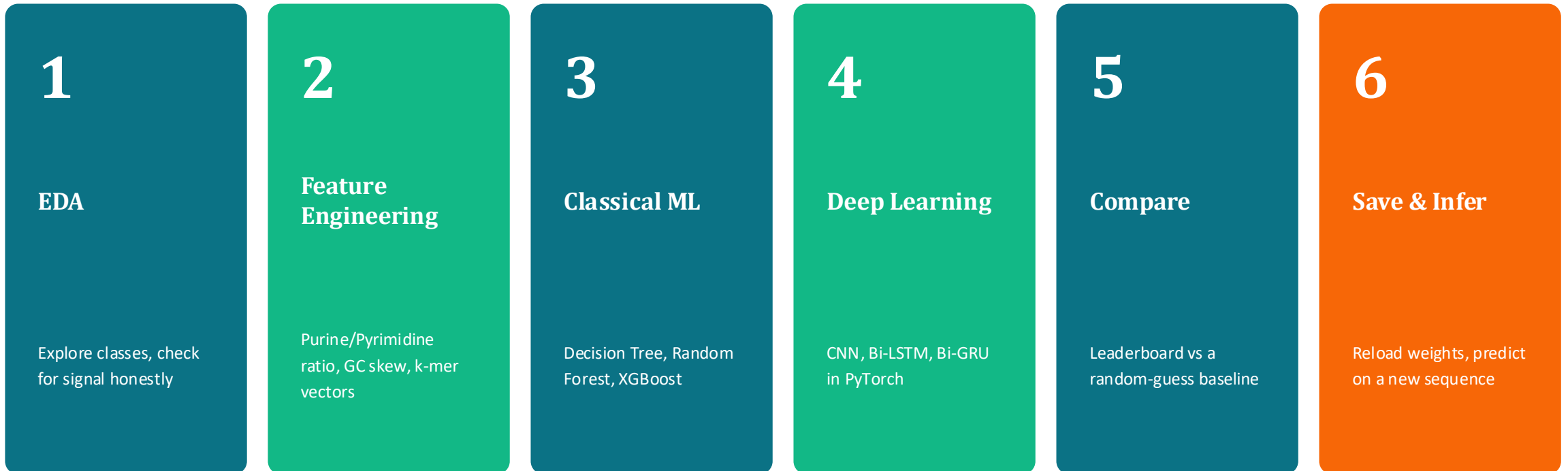
The Dataset: 3,000 Synthetic DNA Sequences

Sample_ID	Sequence	GC_Content	kmer_3_freq	Mutation_Flag	Class_Label	Disease_Risk
SAMPLE_1	CTTTCGGGATAC...	50.0	0.986	0	Bacteria	High
SAMPLE_2	TTGACCAAATTT...	45.0	0.486	1	Virus	Medium
SAMPLE_4	ACTACGCGGACA...	55.0	0.404	0	Human	Medium
SAMPLE_6	GCGGCTTGTTAT...	45.0	0.495	1	Plant	Low

- **Class_Label (target):** Bacteria / Human / Plant / Virus — a 4-class classification problem
- **GC_Content / AT_Content:** % of sequence that is G/C vs A/T
- **kmer_3_freq:** Average frequency of 3-letter sub-patterns
- **Mutation_Flag / Disease_Risk:** Binary mutation marker, and Low/Medium/High disease risk

3,000 rows · 100-letter sequences · balanced across 4 classes · from GitHub (kaopanboonyuen/CP020003_ArtificialIntelligence_2026s1)

The End-to-End Pipeline



Same 6-step recipe as any real ML project — the only thing that changes week to week is the data and the models.



Decision Tree — "20 Questions" for Data

Asks a series of yes/no questions about the features, splitting the data at each step, until it reaches a leaf that predicts a class. Easy to interpret, but a single deep tree can overfit.

Parameter	What It Controls	How To Tune It
<code>max_depth</code>	Max number of questions the tree can ask before predicting	Lower (3-5) = simpler, less overfit. Higher (15+) = can memorize training data
<code>min_samples_split</code>	Min samples a node needs before it can split further	Higher values (20-50) stop splitting on tiny, noisy groups
<code>min_samples_leaf</code>	Min samples allowed in a final leaf	Prevents a leaf built from just 1-2 examples — classic overfit sign
<code>criterion</code>	Formula used to pick the "best" split — gini or entropy	Both usually similar; entropy is a bit more expensive to compute
<code>random_state</code>	The random seed	Always set it — makes results reproducible, run after run

 **Pro tip:** if training accuracy $\gg$ test accuracy, that's overfitting — lower `max_depth` or raise `min_samples_leaf` first.



Random Forest — Ask a Crowd, Take a Vote

Trains many Decision Trees, each on a random subset of data AND features, then lets them vote. This "wisdom of the crowd" generalizes far better than any single tree.

Parameter	What It Controls	How To Tune It
<code>n_estimators</code>	How many trees are in the forest	More trees (100→500) = steadier accuracy, but slower. Returns diminish after ~200-500
<code>max_depth</code>	Max depth of each individual tree	Forests resist overfitting better than one tree, so you can usually go deeper
<code>max_features</code>	How many features each tree can consider per split	Restricting this (e.g. "sqrt") forces trees to differ — key to the crowd effect
<code>min_samples_leaf</code>	Min samples per leaf (same idea as Decision Tree)	Same anti-overfitting logic as a single tree
<code>n_jobs</code>	How many CPU cores to train in parallel	Set to -1 to use every core — free speed, zero accuracy cost

 **Pro tip:** plot test accuracy at `n_estimators` = 100, 300, 600 — once the curve flattens, you've found the sweet spot.



XGBoost — Trees That Learn From Mistakes

Also builds many trees, but sequentially: each new tree is trained specifically to fix the previous trees' errors. One of the most consistently high-performing methods on tabular data.

Parameter	What It Controls	How To Tune It
<code>n_estimators</code>	Number of boosting rounds (trees added in sequence)	More rounds can keep improving accuracy, but risks overfitting if <code>learning_rate</code> is high
<code>max_depth</code>	Depth of each individual boosted tree	Kept shallow on purpose (3-6) — boosting combines many weak learners, not a few strong ones
<code>learning_rate (eta)</code>	How much each new tree corrects previous mistakes	Lower (0.01-0.05) = more careful & accurate, needs more <code>n_estimators</code>
<code>subsample</code>	Fraction of rows randomly sampled per round	0.7-0.9 adds helpful randomness that reduces overfitting
<code>colsample_bytree</code>	Fraction of features randomly sampled per tree	Same idea as <code>subsample</code> , but for columns

 **Pro tip:** the classic recipe: lower the `learning_rate`, raise `n_estimators` — e.g. 0.1/100 → 0.02/500 for a bit more accuracy.



Neural Network Building Blocks

Every network in this course is built from the same handful of reusable layer types, stacked in different orders.

Layer	What It Does	Key Parameter To Tune
Linear / Dense	A fully-connected layer: every input connects to every output — the classic "weighted sum" layer	out_features — how many neurons; more = more capacity
Conv1d	Slides a small window across a sequence to detect local patterns (motifs)	kernel_size — how wide each filter's "view" is
ReLU (activation)	Adds non-linearity — without it, stacking layers would collapse into one linear function	none to tune — just always include one after each linear/conv layer
Dropout	Randomly "switches off" a fraction of neurons during training, to fight overfitting	p — fraction dropped; 0.2-0.5 is a safe range
Global Max Pool	Collapses a whole sequence of values down to its single strongest signal	none — but decides WHERE the network "reads from"

Last time's CNN recipe: Conv1d (finds candidate motifs) → ReLU → Global Max Pool ("did it appear anywhere?") → Dropout → Linear → Linear (class scores). Position-invariant motif detection, DeepBind-style.



LSTM — Long Short-Term Memory

Reads a sequence one letter at a time, left to right, keeping a running "memory" (hidden state) updated by 3 internal gates.

Forget Gate

Decides what old information to throw away

Input Gate

Decides what new information to add to memory

Output Gate

Decides what to output based on current memory

Best suited for: sequential / order-dependent data where long-range context matters — text, time series, and (here) DNA sequences where a motif could sit anywhere among 100 letters.

Parameter	What It Controls	Tuning Tip
<code>hidden_dim</code>	Size of the memory vector carried at each step	Start at 32; raise if underfitting
<code>bidirectional</code>	Reads forward AND backward, combines both	Turn on — halves the "memory distance" needed
<code>embed_dim</code>	Size of the learned vector per letter (A/C/G/T)	4-16 is plenty for a 4-symbol alphabet



GRU — Gated Recurrent Unit

A lighter cousin of the LSTM — just 2 gates and a single hidden state instead of 3 gates and two memory states.

Update Gate

Decides how much of the old memory to keep vs. replace

Reset Gate

Decides how much past information to forget when computing new memory

	LSTM	GRU
Gates	3 (forget, input, output)	2 (update, reset)
Memory states	2 (hidden + cell)	1 (hidden only)
Training speed	Slower (more parameters)	Faster, lighter

Best suited for: the same sequential tasks as LSTM, when you want a faster-to-train, lighter alternative with comparable accuracy.

How to Design Your Own Architecture

1

What shape is my input?

Categories in a sequence → Embedding (RNN) or one-hot + Conv1d (CNN). Flat numbers → Linear.

2

Does position matter, or just presence?

"Anywhere in the input?" → pooling (max/avg). "What happened at the end?" → final hidden state.

3

How much capacity do I need?

Start small. Only add filters/hidden units if BOTH train and test accuracy are low (underfitting).

4

What's my output shape?

N classes → Linear(..., N), no activation (softmax lives inside CrossEntropyLoss).

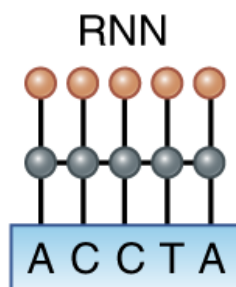
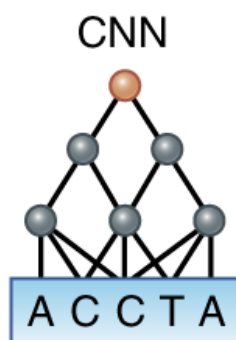
5

Where might it overfit?

Add Dropout before the final classifier layers; watch the loss/accuracy curves to catch it early.

A primer on deep learning in genomicsJames Zou , Mikael Huss, Abubakar Abid, Pejman Mohammadi, Ali Torkamani & Amalio Telenti *Nature Genetics* 51, 12–18 (2019) | [Cite this article](#)[Save article](#)76k Accesses | 907 Citations | 241 Altmetric | [Metrics](#)**a Curate data**

Sequence	Label	
ACCTA	1	Training
ATCTC	1	
TCATT	0	
GAACT	0	
CGGAT	1	Validation
ACAAC	0	
TGCTA	1	Test
AGCCC	0	

b Select architecture, train

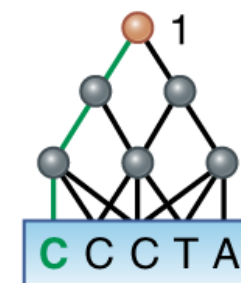
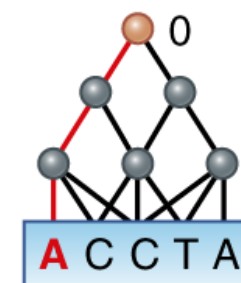
● Internal unit ● Output

c Evaluate

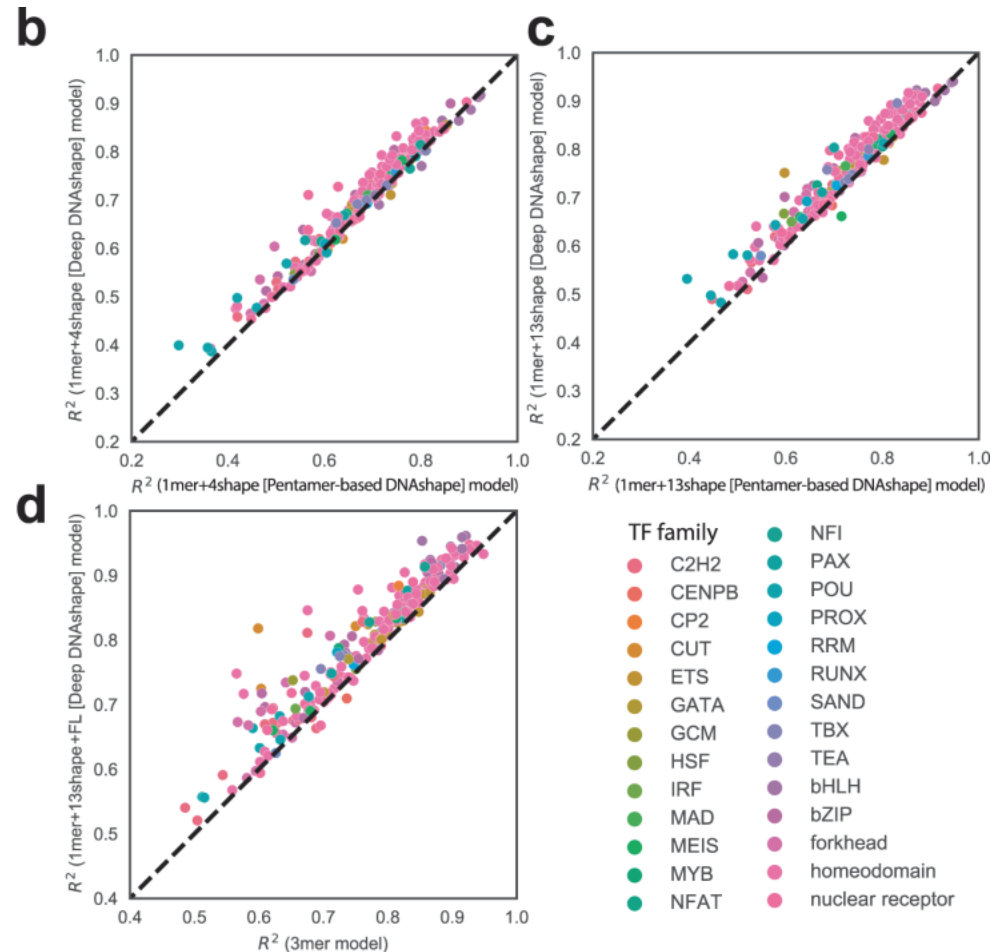
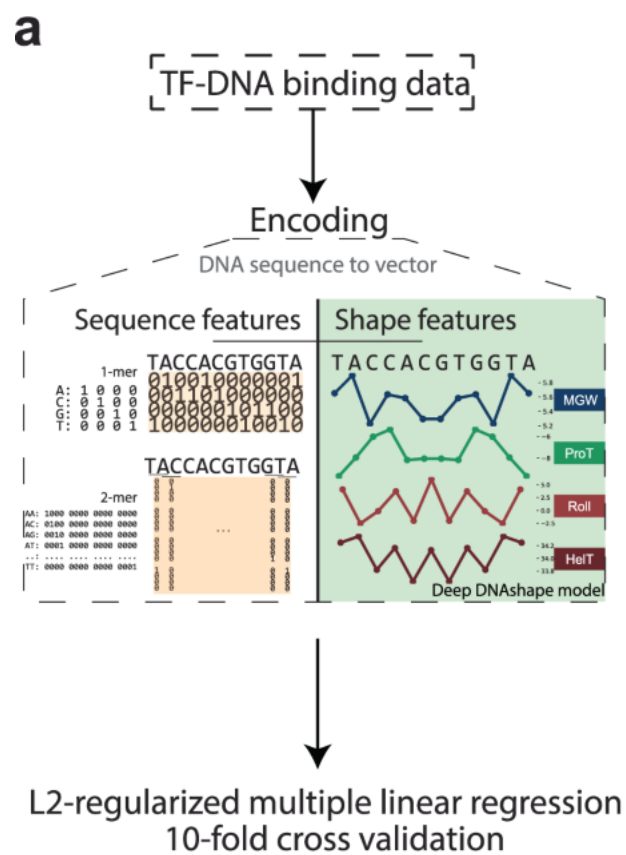
	Predicted	
	+	-
Actual	+	TP FN
	-	FP TN

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

d Interpret

Feature importance



nature communications

[Explore content](#) [About the journal](#) [Publish with us](#)

[nature](#) > [nature communications](#) > [articles](#) > [article](#)

Article | [Open access](#) | Published: 09 February 2024

Predicting DNA structure using a deep learning method

[Jinsen Li](#), [Tsu-Pei Chiu](#) & [Remo Rohs](#)

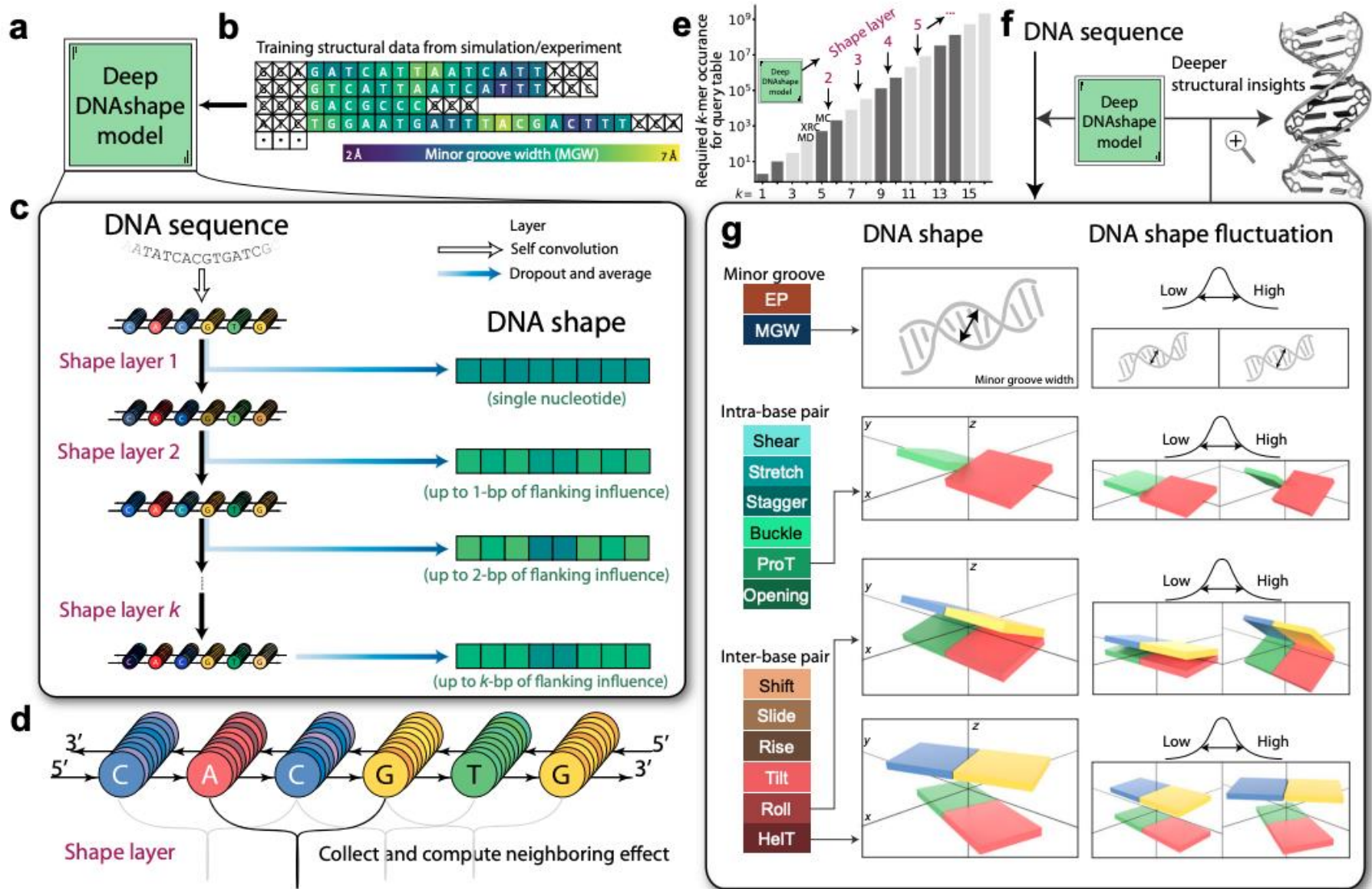
[Nature Communications](#) **15**, Article number: 1243 (2024) | [Cite this article](#)

[Save article](#)

29k Accesses | 84 Citations | 27 Altmetric | [Metrics](#)

Article

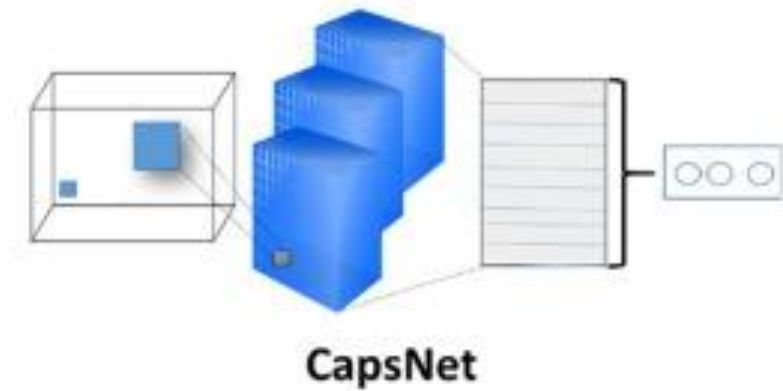
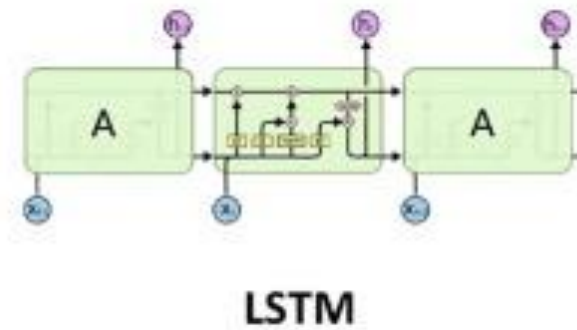
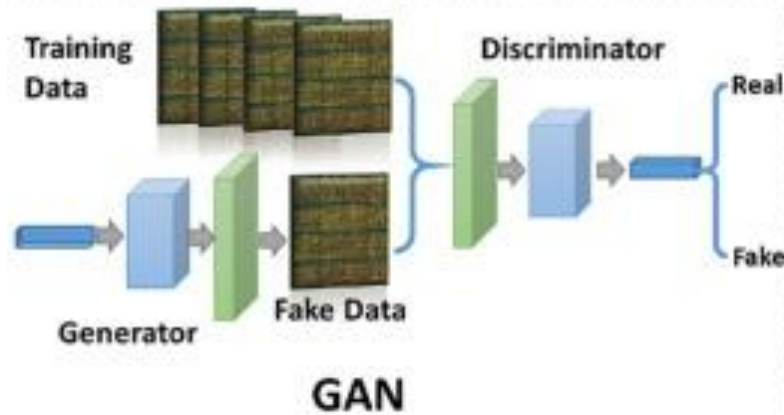
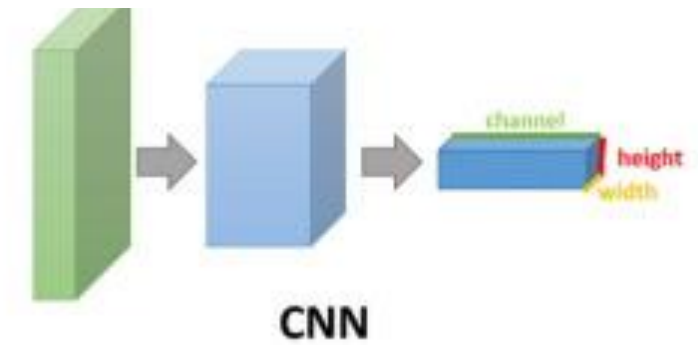
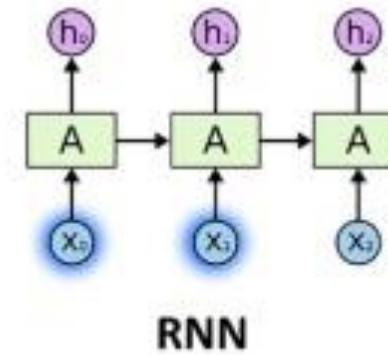
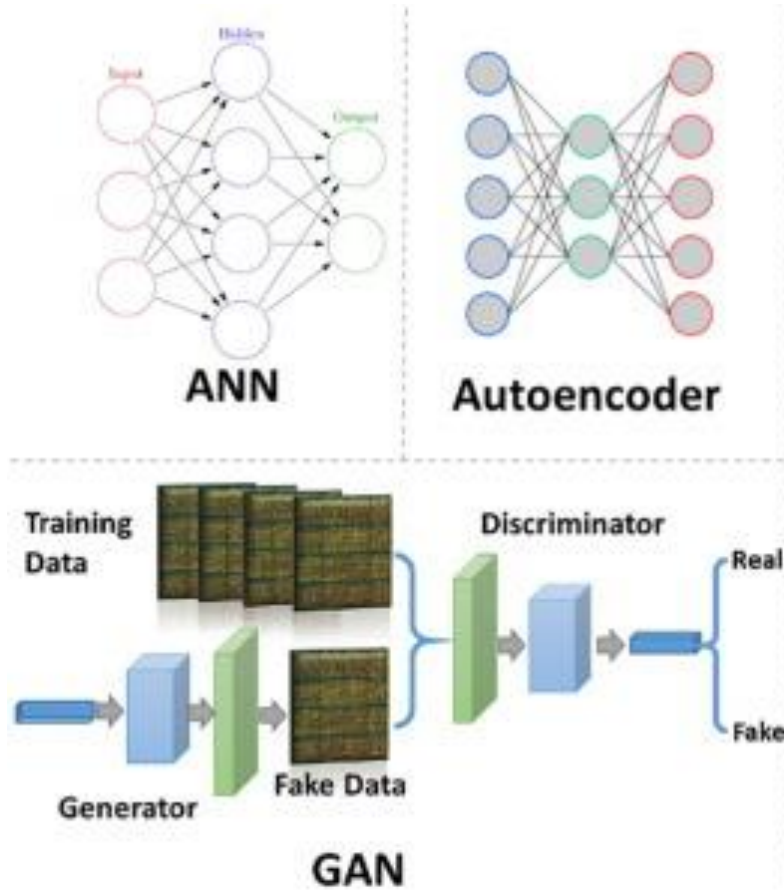
<https://doi.org/10.1038/s41467-024-45191-5>



Review

Deep learning models in genomics; are we there yet?

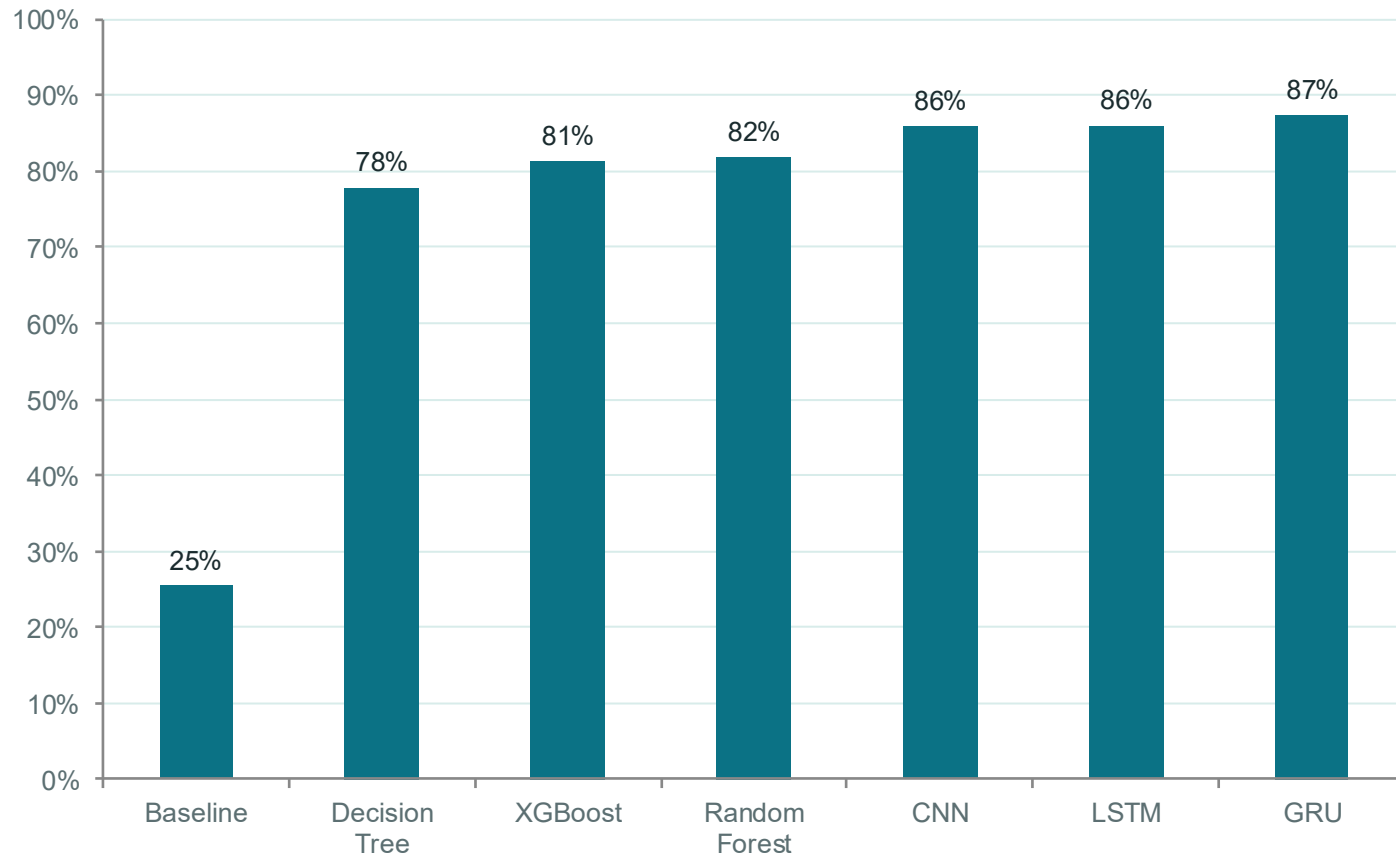
Lefteris Koumakis ✉



25

RESULTS

Model Leaderboard: 6 Models vs. Baseline



Key Insight

Every model clears the 25.4% baseline by a wide margin.

The win wasn't a fancier algorithm — it was matching feature representation (k-mer vectors) and architecture (global/max pooling) to the actual structure of the problem: "does this motif appear anywhere?"

Winner this run: GRU at 87.3%

Baseline = always predicting the majority class (25.4%) — the honest yardstick every model must beat.

Key Takeaways

1

DNA is just a string

A 4-letter alphabet {A,C,G,T} — everything else builds on that one idea

2

Always check a baseline first

An accuracy number means nothing without something honest to compare it to

3

Features + architecture > fancier models

Richer k-mer vectors and position-invariant pooling beat throwing more layers at the problem

4

CNNs and RNNs both do motif-finding

Global max pooling (CNN) and max-pool-over-timesteps (Bi-LSTM/GRU) solve the same "anywhere?" question

5

Reproducibility matters

stratify, fit-on-train-only scaling, and random_state are small habits that prevent big mistakes

6

A model isn't done until it's reusable

Save weights, reload them, and run real inference on brand-new data



This Week's Challenge: DNA Methylation

A new, real bioinformatics dataset: CpG-site methylation data from the Illumina platform used in real epigenetics research.

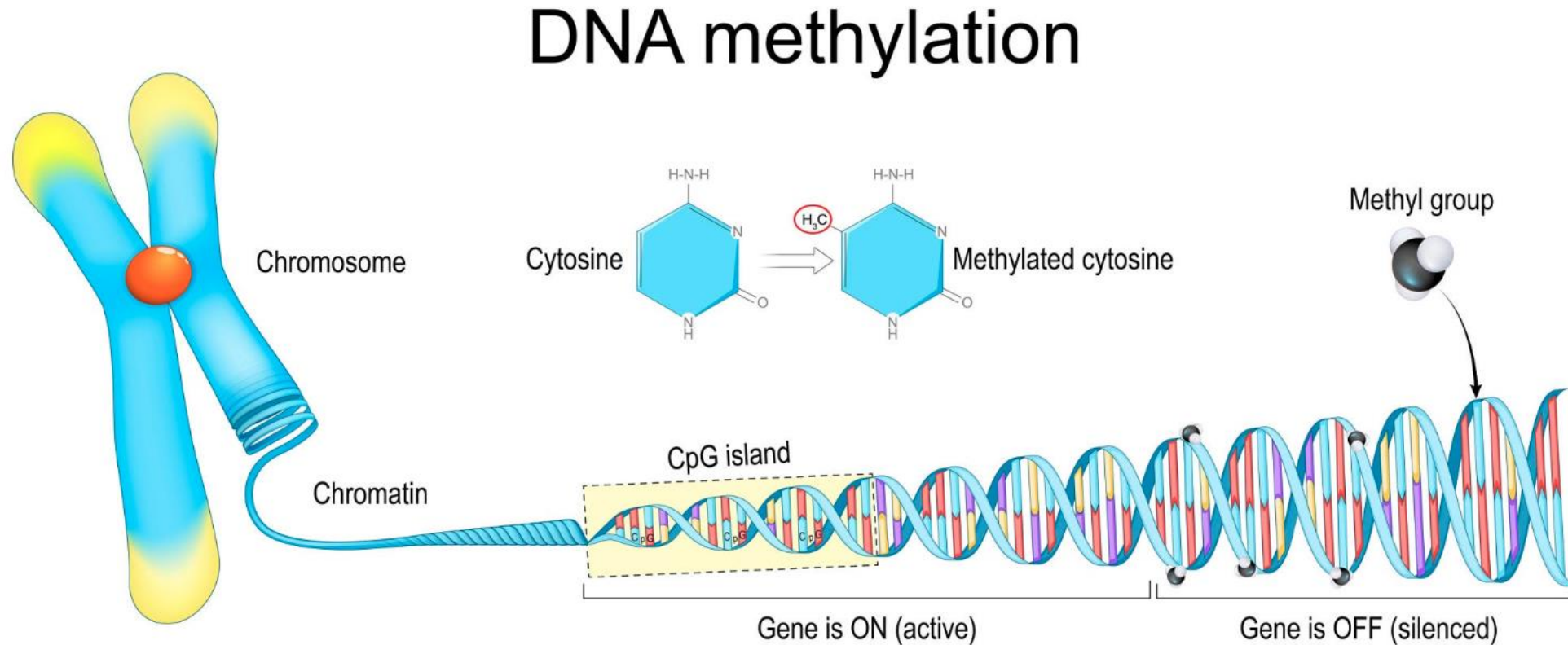
Id	CHR	UCSC_RefGene_Group	Relation_to_CpG_Island	Beta
cg00001583	1	Body;Body	Island	0
cg00002719	1	TSS200	Island	0

- **Beta (target):** Methylation level at that CpG site, 0 (unmethylated) to 1 (fully methylated)
- **UCSC_RefGene_Group:** Where the site sits relative to a gene (Body, TSS200, ...)
- **Relation_to_CpG_Island:** Whether the site is inside a CpG Island or nearby
- **Forward_Sequence / seq:** The real surrounding DNA sequence — your feature-engineering playground

Your task: engineer features from Forward_Sequence / seq (GC content, k-mer vectors, CpG counts...) and try predicting Beta — or bucket it into Low/Medium/High and reuse this week's classifiers!

This Week's Challenge: DNA Methylation

A new, real bioinformatics dataset: CpG-site methylation data from the Illumina platform used in real epigenetics research.



Have fun with DNA + AI 🧬

Let's Build End-to-End Models

Questions before we open Colab?

`github.com/kaopanboonyuen/CP020003_ArtificialIntelligence_2026s1`

